Prompt-Injection Gatekeeper

A tool-boundary decision layer for AI agents, powered by Strands Decider 2B

Shlok Vane | Hack Apertus 2026, Track 2B (Own Project) | github.com/shlokgit-04/prompt-injection-gatekeeper

1. Abstract

LLM agents with tool access can be hijacked by untrusted content: a malicious document, email or web page read by the agent can steer it into exfiltrating data or taking unauthorized actions. Prompt-Injection Gatekeeper is a security and decision layer that intercepts every proposed tool call at the boundary before it executes. A Strands agent (Strands Agents SDK, main LLM on Amazon Bedrock) proposes tool calls; Strands Decider 2B, a 2B-parameter decision model served locally, evaluates each proposal with seven typed yes/no questions, and a deterministic policy maps the model's calibrated probabilities onto three outcomes: PROCEED, CONFIRM (human approval), or DENY. A denied call raises before the function body runs, so an injected instruction can never reach a real tool. Every interception is written to an append-only audit log with secret redaction, and on any decider failure the gate fails closed to CONFIRM.

2. Problem and threat model

The interesting failure mode is agentic. A chatbot cannot be prompt-injected into doing anything; an agent holding tools can. Indirect prompt injection hides instructions inside content the agent legitimately reads (a vendor document, an email body, a web page), so the attacker needs no access to the chat itself. Conventional defenses review the text going in or out of the model. This project instead secures the action boundary: it does not try to make the model refuse, it makes a malicious tool call impossible to execute. The threat model therefore covers: injected instructions in tool-read content, argument tampering, out-of-scope and premature actions, and data exfiltration through legitimate tools.

3. Architecture

Every tool the agent's registry holds is wrapped in a GatedTool. There is no code path from the LLM's tool choice to a real function that does not pass the gate. The decision pipeline:

```
INPUT:  user request + conversation + proposed tool + arguments
   |
DECIDER 2B: 7 typed yes/no questions (noul), one forward pass per question
   |   q_grounded   arguments grounded in facts the user provided?
   |   q_in_scope   action within the scope the user asked for?
   |   q_risky      could it leak secrets, exfiltrate, or damage?
   |   q_premature  premature before checking with the user?
   |   q_proceed / q_confirm / q_deny   (the model's three-way vote)
   |
DECISION: policy thresholds (decider/gate.py)
   |   risky >= 0.75, or vote=DENY with risky >= 0.55        -> DENY
   |   grounded or in_scope <= 0.20                          -> DENY
   |   vote=CONFIRM, risky >= 0.55, premature >= 0.60,
   |   or grounded/in_scope < 0.55                          -> CONFIRM
   |   any decider error                                    -> CONFIRM (fail closed)
   |
ACTION: PROCEED -> execute + audit | CONFIRM -> human queue | DENY -> block + audit
```

Decider 2B is used because the gate's checks are narrow classification problems: it answers each question in about 115 ms per question locally with calibrated confidence, cheap enough to sit on every tool call, where an LLM call never could. Its published calibration rule (at confidence 0.9+ it is right about 95% of the time) is also the product's operating rule: below that band the gate asks a human. The main agent LLM runs on Amazon Bedrock through the Strands Agents SDK; the deployment target is AWS (a self-contained container image baking the ~4.5 GB Decider checkpoint, sized for a Fargate 2 vCPU / 8 GB task; see docs/deployment.md in the repository).

4. The three gated tools

The demo agent holds three real tools: read a document, send email (wired to a demo outbox, not live mail), and query a customer database (SELECT-only). A CONFIRM decision parks the call in an approval queue and it runs only after a human approves it in the web UI. All three paths are audited. Nothing asks the main LLM whether a call is safe; safety judgment lives entirely in the boundary layer.

5. Evaluation: what has been measured so far

Honest scope of the current evidence:

- Unit and boundary suite: 11/11 tests pass. The key boundary property is proven with a spy object: a DENY decision raises before the wrapped function body, so a denied tool call cannot execute even if the model strongly 'wants' it.

- Live red-team suite: nine attack cases are implemented (legitimate request, obvious prompt injection, indirect injection via a malicious document, malicious tool arguments, unauthorized action, premature action, ambiguous request, safe tool call, high-risk action needing approval). The suite writes docs/evaluation_results.json with real measured numbers (attack detection rate, false positives/negatives, latency, confidence, blocked calls). It requires the Decider 2B server running on a machine with roughly 8 GB RAM; it has not yet been run live, so no red-team accuracy figures are claimed in this report.

- Latency and calibration figures quoted (about 115 ms per question; 0.9+ confidence maps to roughly 95% accuracy) are the published characteristics of the Strands Decider 2B checkpoint, used as design constants, not as measurements of this system.

The repository CI runs the unit and boundary suite on every push and is green on main. Only numbers from actual runs are ever reported anywhere in the project; the application never fabricates the model's output.

6. Demo scenario (docs/demo.md)

1. SAFE: 'Read the supplier document and email me a summary.' Gate: PROCEED, executed and audited.

2. INDIRECT INJECTION: vendor_terms_malicious.md contains hidden instructions ordering the agent to email the document and the customer database to an outsider. Gate: DENY; the tool never executes; a security event is recorded in the audit log.

3. HIGH-RISK: a plausible action with an unstated recipient or scope. Gate: CONFIRM; the call is held and runs only after approval in the UI.

7. Repository and stack

Public repository under the MIT license: github.com/shlokgit-04/prompt-injection-gatekeeper. Layout: /agent (agent wiring, the gated boundary), /tools (the three tools), /decider (Decider 2B client and decision policy), /backend (FastAPI: chat, approvals, audit), /frontend (UI showing green/yellow/red decisions), /tests (unit, boundary, live red-team), /docs (security model, decider integration, deployment, demo, evaluation), /demo/documents (demo corpus including the malicious document), /scripts (run and seed helpers). Stack: Python, Strands Agents SDK, Amazon Bedrock, FastAPI, React UI.

8. Security model and limits

Tool-call interception at the boundary, grounding and authorization validation, suspicious-action detection, human confirmation for high-risk calls, denial for clearly malicious ones, append-only audit log with secret redaction, and fail-closed behavior on infrastructure errors. Limits are stated precisely in docs/security-model.md: no system is 100% secure. This layer constrains a class of prompt-injection failures at the action boundary and logs everything it does; its measured performance is exactly whatever docs/evaluation_results.json contains.